

```
--- djangoapi\scripts\crop\plantas\plantasP00.txt ---
```

```
from psycopg.rows import dict_row
from myLib.gestor import tablasP00
```

```
class PlantasP00(tablasP00):
```

```
    # INSERT
```

```
    def insert(self, d: dict):
```

```
        wkt = d['geom_wkt']
```

```
        if not self.geom_isValid(wkt):
```

```
            print("Error: La geometría no es válida")
```

```
            self.disconnect()
```

```
            return None
```

```
        if self.has_interior_intersection('plantas', wkt):
```

```
            print("Error: El punto intersecta el interior de uno existente.")
```

```
            self.disconnect()
```

```
            return None
```

```
        geom_final = self.snap_to_grid(wkt)
```

```
        variedad = d['variedad']
```

```
        estado_salud = d['estado_salud']
```

```
        fecha_cosecha_est = d['fecha_cosecha_est']
```

```
        cons = """
```

```
            INSERT INTO plantas
```

```
                (variedad, estado_salud, fecha_cosecha_est, geom)
```

```
            VALUES
```

```
                (%s, %s, %s, ST_GeomFromText(%s, %s))
```

```
            RETURNING id
```

```
        """
```

```
        try:
```

```
            self.cur.execute(cons, [variedad, estado_salud, fecha_cosecha_est,
                                     geom_final, self.epsg])
```

```
            self.conn.commit()
```

```
            new_id = self.cur.fetchone()[0]
```

```
            print(f"Planta '{variedad}' insertada con id: {new_id}")
```

```
            return new_id
```

```
        except Exception as e:
```

```
            print(f"Error en inserción de planta: {e}")
```

```
            self.conn.rollback()
```

```
            return None
```

```
        finally:
```

```
            self.disconnect()
```

```
    # SELECTALL
```

```
    def selectAll(self, d: dict):
```

```
        id_min = d['id_min']
```

```

cons = """
    SELECT COUNT(*) AS total
    FROM plantas
    WHERE id > %s
"""
try:
    self.cur.execute(cons, [id_min])
    total = self.cur.fetchone()[0]
    print(f"{total} planta(s) encontrada(s).")
    return total

except Exception as e:
    print(f"Error en selectAll de plantas: {e}")
    return 0

finally:
    self.disconnect()

```

SELECTASTUPLE

```

def selectAsTuple(self, d: dict):
    id_min = d['id_min']
    cons = """
        SELECT id, variedad, estado_salud, fecha_cosecha_est,
               ST_AsText(geom) AS geom_wkt
        FROM plantas
        WHERE id > %s
    """
    try:
        self.cur.execute(cons, [id_min])
        rows = self.cur.fetchall()
        print(f"{len(rows)} planta(s) encontrada(s).")
        return rows

    except Exception as e:
        print(f"Error en selectAsTuple de plantas: {e}")
        return []

    finally:
        self.disconnect()

```

SELECTASDICT

```

def selectAsDict(self, d: dict):
    id_min = d['id_min']
    cons = """
        SELECT id, variedad, estado_salud, fecha_cosecha_est,
               ST_AsText(geom) AS geom_wkt
        FROM plantas
        WHERE id > %s
    """
    try:
        self.cur = self.conn.cursor(row_factory=dict_row)
        self.cur.execute(cons, [id_min])
        rows = self.cur.fetchall()

```

```

        print(f"{len(rows)} planta(s) encontrada(s).")
        return rows

    except Exception as e:
        print(f"Error en selectAsDict de plantas: {e}")
        return []

    finally:
        self.disconnect()

# UPDATE
def update(self, d: dict):
    wkt = d['geom_wkt']

    if not self.exists_by_id('plantas', d['id']):
        print(f"Error: No existe la planta con id {d['id']}.")
        self.disconnect()
        return None

    if not self.geom_isValid(wkt):
        print("Error: La geometria no es valida.")
        self.disconnect()
        return None

    geom_final = self.snap_to_grid(wkt)

    if self.has_interior_intersection_excluding_id('plantas', geom_final, d['id']):
        print("Error: La planta intersecta el interior de una existente.")
        self.disconnect()
        return None

    cons = """
        UPDATE plantas
        SET
            variedad          = %s,
            estado_salud       = %s,
            fecha_cosecha_est  = %s,
            geom                = ST_GeomFromText(%s, %s)
        WHERE id = %s
    """
    try:
        self.cur.execute(cons, [
            d['variedad'], d['estado_salud'], d['fecha_cosecha_est'],
            geom_final, self.epsg, d['id']
        ])
        self.conn.commit()
        print(f"Planta actualizada: {self.cur.rowcount} fila(s).")

    except Exception as e:
        print(f"Error en update de plantas: {e}")
        self.conn.rollback()

    finally:

```

```
        self.disconnect()

# DELETE
def delete(self, d: dict):
    cons = "DELETE FROM plantas WHERE id = %s"
    try:
        self.cur.execute(cons, [d['id']])
        self.conn.commit()
        print(f"Plantas eliminadas: {self.cur.rowcount} fila(s).")

    except Exception as e:
        print(f"Error en delete de plantas: {e}")
        self.conn.rollback()

    finally:
        self.disconnect()
```